

BASH SCRIPTING FUNDAMENTALS

Files, Pipes & Redirection

Streams, pipelines, and gluing Unix tools together

Merit Advisory

Bash Scripting Fundamentals Bootcamp

Beginner-friendly lecture materials

July 2026



Agenda

- 1 Section 1: Stream Fundamentals — 8 topics
- 2 Section 2: Output Redirection — 8 topics
- 3 Section 3: stderr and Silence — 8 topics
- 4 Section 4: Pipes and Pipelines — 8 topics
- 5 Section 5: tee and Logging — 8 topics
- 6 Section 6: Here Input and Substitution — 8 topics
- 7 Section 7: File Descriptors and exec — 8 topics
- 8 Section 8: Temporary Files and Cleanup — 8 topics

9 Section 9: Batching and File Streams — 8 topics

1 Section 10: Filesystem Helpers — 8 topics

0

1 Section 11: Disk Awareness and Checkpoint — 8 topics

1

1 Section 1

Stream Fundamentals

1.1 stdout carries normal output

- stdout is file descriptor 1.

Print normal output

```
printf 'ready\n'
```

- Commands write useful results to stdout by default.

Redirect stdout

```
printf 'ready\n' > status.txt
```

1.2 stdin supplies input

- stdin is file descriptor 0.

Read from a file

```
wc -l < names.txt
```

- Many filters read stdin when no file is named.

Read from a pipe

```
printf 'a\nb\n' | wc -l
```

1.3 stderr carries diagnostics

- stderr is file descriptor 2.

Show an error stream

```
ls missing.txt
```

- Errors and warnings should not mix with normal data.

Capture only errors

```
ls missing.txt 2> errors.log
```

1.4 File descriptors identify streams

- A file descriptor is a small number attached to an open stream.

Name the standard fds

```
printf 'stdin=0 stdout=1 stderr=2\n'
```

- Bash starts every command with descriptors 0, 1, and 2.

Send fd 1 to a file

```
date 1> now.txt
```

1.5 Streams are not always files

- A stream may point at a terminal, pipe, socket, or file.

Terminal target

```
printf 'hello\n'
```

- The command usually does not need to know the target type.

File target

```
printf 'hello\n' > hello.txt
```

1.6 Input redirection feeds stdin

- The < operator opens a file for fd 0.

Count file lines

```
wc -l < app.log
```

- The command reads the file as standard input.

Sort redirected input

```
sort < names.txt
```

1.7 Output redirection captures stdout

- The > and >> operators target fd 1 unless another fd is named.

Capture a listing

```
ls > listing.txt
```

- Output redirection happens before the command body runs.

Append a listing

```
ls >> listing.txt
```

1.8 Exit status is separate from streams

- A command can print text and still fail.

Quiet success

```
true > out.txt
```

- A command can be quiet and still succeed.

Failed command with stderr

```
grep needle missing.txt 2> err.txt
```

2 Section 2

Output Redirection

2.1 Overwrite with >

- > creates the target file if needed.

Create a report

```
date > report.txt
```

- > truncates an existing target before writing.

Replace the report

```
hostname > report.txt
```

2.2 Append with >>

- >> creates the target file if needed.

Append one line

```
date >> run.log
```

- >> keeps existing content and writes at the end.

Append command output

```
uptime >> run.log
```

2.3 Clobbering is easy

- A mistaken > can erase a file before the command starts.

Dangerous overwrite

```
printf 'new\n' > notes.txt
```

- Review important output paths before pressing Enter.

Safer append

```
printf 'new\n' >> notes.txt
```

2.4 noclobber protects overwrites

- `set -o noclobber` makes `>` refuse existing files.

Enable protection

```
set -o noclobber
```

- `>|` intentionally overrides `noclobber` for one redirect.

Force one overwrite

```
printf 'ok\n' >| output.txt
```

2.5 Redirect a command group

- Braces group several commands in the current shell.

Write a small report

```
{ date; hostname; } > report.txt
```

- One redirect can capture the whole group.

Append a grouped log

```
{ echo start; date; } >> run.log
```

2.6 Create or truncate with :

- : is a do-nothing command that succeeds.

Create empty file

```
: > empty.txt
```

- Redirecting : can create an empty file.

Reset a log

```
: > run.log
```

2.7 sudo and redirects are separate

- Redirection is performed by your current shell.

Often fails

```
sudo echo ok > /etc/example.conf
```

- sudo before a command does not sudo the redirect.

Use tee

```
echo ok | sudo tee /etc/example.conf
```

2.8 Redirect to predictable paths

- Relative redirect paths depend on the current directory.

Use a variable

```
log='logs/run.log'  
date >> "$log"
```

- Variables make output destinations easier to audit.

Write beside a script

```
out="$PWD/output.txt"  
printf 'ok\n' > "$out"
```

3 Section 3

stderr and Silence

3.1 Redirect stderr with 2>

- 2> changes file descriptor 2 only.

Capture errors

```
grep root missing.txt 2> errors.log
```

- stdout still goes wherever it was already going.

Keep stdout visible

```
find /root -maxdepth 1 2> denied.log
```

3.2 Separate stdout and stderr

- You can send stdout and stderr to different files.

Two files

```
cmd > output.log 2> errors.log
```

- This keeps results separate from diagnostics.

Find example

```
find . -name '*.sh' > files.txt 2> find.err
```

3.3 Merge stderr into stdout with 2>&1

- 2>&1 makes fd 2 point to the current fd 1 target.

Combined file

```
cmd > combined.log 2>&1
```

- Order matters because redirects are processed left to right.

Combined pipe

```
cmd 2>&1 | tee combined.log
```

3.4 Redirection order changes results

- `cmd >file 2>&1` sends both streams to file.

Both to file

```
cmd > all.log 2>&1
```

- `cmd 2>&1 >file` sends stderr to the old stdout.

stderr stays visible

```
cmd 2>&1 > out.log
```

3.5 Use &> for both streams

- &> file redirects stdout and stderr together in Bash.

Combined overwrite

```
cmd &> all.log
```

- It is shorter than > file 2>&1.

Combined append

```
cmd &>> all.log
```

3.6 Discard stdout with >/dev/null

- /dev/null accepts data and throws it away.

Ignore normal output

```
curl -s https://example.com >/dev/null
```

- >/dev/null silences normal output only.

Keep errors visible

```
find . -name '*.tmp' >/dev/null
```

3.7 Discard stderr with 2>/dev/null

- 2>/dev/null silences diagnostics only.

Hide permission errors

```
find / -name passwd 2>/dev/null
```

- stdout remains available for data.

Keep matches

```
grep root /etc/* 2>/dev/null
```

3.8 Silence both streams

- Redirect stdout to /dev/null first.

Portable Bash pattern

```
cmd >/dev/null 2>&1
```

- Then point stderr at the same place.

Bash shortcut

```
cmd &>/dev/null
```

4 Section 4

Pipes and Pipelines

4.1 Pipe with |

- | connects stdout from the left command to stdin of the right command.

Count entries

```
ls | wc -l
```

- Pipes avoid temporary files for simple transformations.

Filter output

```
ps aux | grep ssh
```

4.2 Pipelines read left to right

- Each stage receives the previous stage's stdout.

Filter then count

```
printf 'a\nb\n' | grep a | wc -l
```

- Think of a pipeline as data moving through filters.

List then sort

```
ls | sort
```

4.3 Filters transform streams

- A filter reads stdin and writes stdout.

Sort a stream

```
printf 'b\na\n' | sort
```

- Classic filters include sort, cut, grep, sed, and awk.

Take a column

```
printf 'a:b\n' | cut -d: -f1
```

4.4 Pipe to commands, redirect to files

- Use | when another command should read the data.

Pipe to a command

```
ls | wc -l
```

- Use > or >> when a file should store the data.

Save pipeline output

```
ls | sort > files.txt
```

4.5 Pipeline status normally uses the last command

- Bash returns the status of the last pipeline command by default.

Last command wins

```
false | true  
echo $?
```

- An early failure can be hidden by a later success.

Fail at the end

```
true | false  
echo $?
```

4.6 pipefail previews safer pipelines

- set -o pipefail makes a pipeline fail when any stage fails.

Enable pipefail

```
set -o pipefail
```

- It is commonly paired with strict script settings.

Check a pipeline

```
set -o pipefail  
false | true
```

4.7 Buffering can delay output

- Programs may buffer output differently in pipes than terminals.

Live log pipeline

```
tail -f app.log | grep ERROR
```

- Buffered output can make live pipelines appear stuck.

Line-buffer grep

```
tail -f app.log | grep --line-buffered ERROR
```

4.8 yes generates repeated input

- yes prints a string repeatedly until stopped.

Generate y

```
yes | head -n 3
```

- It can feed prompts in controlled examples.

Generate a word

```
yes ok | head -n 2
```

5 Section 5

tee and Logging

5.1 tee writes to screen and file

- tee copies stdin to stdout and files.

Save and display

```
make 2>&1 | tee build.log
```

- It lets you watch output while saving it.

Write one line

```
echo ready | tee status.txt
```

5.2 tee -a appends

- tee -a keeps existing file content.

Append output

```
date | tee -a run.log
```

- Append mode is useful for long-running logs.

Append errors too

```
cmd 2>&1 | tee -a all.log
```

5.3 Log and screen together

- A combined log captures what the operator saw.

Capture a run

```
./backup.sh 2>&1 | tee backup.log
```

- 2>&1 before tee includes errors in the log.

Append a run

```
./backup.sh 2>&1 | tee -a backup.log
```

5.4 Append timestamps to logs

- A timestamp gives each log entry context.

Date prefix

```
printf '%s start\n' "$(date +%F_%T)" >> run.log
```

- Use command substitution to generate the time.

Group with timestamp

```
{ date '+%F %T'; echo done; } >> run.log
```

5.5 Use a simple logging function

- A function keeps log formatting consistent.

Define logger

```
log(){ printf '%s %s\n' "$(date +%F_%T)" "$*" >> app.log; }
```

- Append inside the function to avoid repeated redirects.

Call logger

```
log 'backup started'
```

5.6 Split output and error logs

- Separate logs keep successful data apart from diagnostics.

Separate logs

```
./job.sh > job.out 2> job.err
```

- This is useful when stdout is machine-readable.

Append separately

```
./job.sh >> job.out 2>> job.err
```

5.7 Rotate a simple log

- Rotation moves an old log before starting a new one.

Move old log

```
mv app.log "app.$(date +%F_%H%M%S).log"
```

- A timestamped suffix keeps previous runs available.

Start fresh

```
: > app.log
```

5.8 tee affects pipeline status

- A command piped to tee becomes part of a pipeline.

Hidden failure risk

```
false | tee run.log  
echo $?
```

- Without pipefail, the pipeline status can hide the first command.

Safer logging

```
set -o pipefail  
false | tee run.log
```

6 Section 6

Here Input and Substitution

6.1 Here-documents use <<EOF

- A here-document feeds multiple lines to stdin.

Feed cat

```
cat <<EOF  
hello  
EOF
```

- The delimiter marks where the input ends.

Create a file

```
cat > note.txt <<EOF  
hello  
EOF
```

6.2 Quoted here-doc delimiters disable expansion

- <<'EOF' treats the body literally.

Literal dollar

```
cat <<'EOF'  
$HOME  
EOF
```

- Variables and command substitutions are not expanded.

Literal command

```
cat <<'EOF'  
$(date)  
EOF
```

6.3 Tab-stripping here-documents use <<-EOF

- <<- removes leading tab characters from body lines.

Indented body

```
cat <<-EOF
indented
EOF
```

- Only real tabs are stripped, not spaces.

Indented template

```
cat > file.txt <<-EOF
value
EOF
```

6.4 Here-strings use <<<

- A here-string feeds one string to stdin.

Search one value

```
grep root <<< "$USER"
```

- Bash adds a trailing newline.

Count words

```
wc -w <<< "one two"
```

6.5 Process substitution provides input paths

- `<(command)` exposes command output as a temporary path.

Compare streams

```
diff <(sort old.txt) <(sort new.txt)
```

- It lets file-oriented tools read generated streams.

Loop over generated data

```
while read -r x; do echo "$x"; done < <(printf 'a\n')
```

6.6 Process substitution can capture output

- `>(command)` exposes a writable path connected to a command.

Split stdout

```
cmd > >(tee out.log)
```

- Data written to that path becomes stdin for the command.

Compress by path

```
tar cf >(gzip > files.tar.gz) dir
```

6.7 /dev/stdin, /dev/stdout, and /dev/stderr

- These paths refer to the current standard streams.

Read current stdin

```
wc -l /dev/stdin < names.txt
```

- They are useful for commands that require filenames.

Write current stderr

```
echo warning > /dev/stderr
```

6.8 Some commands prefer stdin

- Filters often read stdin when no files are listed.

Filter stdin

```
sort < names.txt
```

- Interactive commands may also read stdin for answers.

Pipe to stdin

```
printf 'one\ntwo\n' | head -n 1
```

7 Section 7

File Descriptors and exec

7.1 exec can redirect the current shell

- exec without a command changes descriptors in the current shell.

Redirect future stdout

```
exec > script.log  
echo logged
```

- A script can send all later stdout to a log.

Redirect future errors

```
exec 2> script.err  
ls missing
```

7.2 exec can capture both streams

- Redirect stdout first, then point stderr at stdout.

Whole-script log

```
exec > run.log 2>&1  
echo start
```

- All following commands share the new destinations.

Append whole script

```
exec >> run.log 2>&1  
echo next
```

7.3 Save and restore stdout

- Duplicate a descriptor before changing it.

Save stdout

```
exec 3>&1  
exec > part.log
```

- Restore stdout when only part of a script should be captured.

Restore stdout

```
exec 1>&3  
exec 3>&-
```

7.4 fd 3 is a useful extra channel

- Descriptors 3 and above are available for script use.

Open fd 3

```
exec 3> debug.log  
echo debug >&3
```

- An extra fd can keep user messages separate from data.

Close fd 3

```
exec 3>&-
```

7.5 Read from an extra descriptor

- A descriptor can be opened for input with <.

Open input fd

```
exec 3< names.txt  
read -r first <&3
```

- read -u reads from a specific descriptor.

Close input fd

```
exec 3<&-
```

7.6 Append through a descriptor

- Opening a descriptor once can simplify repeated writes.

Open append fd

```
exec 3>> audit.log  
echo start >&3
```

- >> on exec opens the descriptor in append mode.

Write later

```
printf 'done\n' >&3
```

7.7 Redirect a loop or block

- A redirect after done feeds or captures the whole loop.

Read loop input

```
while read -r name; do echo "$name"; done < names.txt
```

- This keeps loop code readable.

Capture block output

```
{ echo one; echo two; } > nums.txt
```

7.8 Descriptor lifetime matters

- A descriptor stays open until closed or the process exits.

Open then close

```
exec 3> temp.log  
exec 3>&-
```

- Long-lived descriptors can hold files open.

Close input fd

```
exec 4< data.txt  
exec 4<&-
```

8 Section 8

Temporary Files and Cleanup

8.1 mktemp creates a safe temporary file

- mktemp creates a unique path and prints its name.

Create temp file

```
tmp=$(mktemp)
printf 'data\n' > "$tmp"
```

- It avoids predictable filenames in shared temp directories.

Show temp path

```
tmp=$(mktemp)
echo "$tmp"
```

8.2 mktemp -d creates a temporary directory

- A temp directory is safer for several related files.

Create temp dir

```
tmpdir=$(mktemp -d)
```

- It gives one cleanup target for a script run.

Use temp dir

```
printf 'x\n' > "$tmpdir/data.txt"
```

8.3 trap cleanup EXIT removes temp files

- An EXIT trap runs when the shell exits.

Trap a file

```
tmp=$(mktemp)  
trap 'rm -f "$tmp"' EXIT
```

- It cleans up on success and many failure paths.

Trap a directory

```
tmpdir=$(mktemp -d)  
trap 'rm -rf "$tmpdir"' EXIT
```

8.4 Keep temp paths private

- Avoid hand-built names like /tmp/app.\$\$.

Use a template

```
tmp=$(mktemp /tmp/app.XXXXXX)
```

- Use mktemp patterns when a suffix or prefix helps.

Private directory

```
tmpdir=$(mktemp -d)  
chmod 700 "$tmpdir"
```

8.5 Write temp then move into place

- Write complete content to a temporary file first.

Prepare temp

```
tmp=$(mktemp)  
generate > "$tmp"
```

- mv within one filesystem is atomic for readers.

Publish result

```
mv "$tmp" output.txt
```

8.6 sponge concept for rewriting input

- Some pipelines need to read a file before rewriting it.

With sponge

```
sort names.txt | sponge names.txt
```

- sponge reads all input before opening the output file.

Without sponge

```
sort names.txt > names.tmp  
mv names.tmp names.txt
```

8.7 sync asks the system to flush writes

- sync requests pending filesystem writes be committed.

Flush all writes

```
sync
```

- It is useful before removing media or testing disk writes.

Write then sync

```
cp image.iso /mnt/usb/  
sync
```

8.8 Checkpoint temporary file habits

- Create temporary paths with mktemp.

Minimal pattern

```
tmp=$(mktemp)  
trap 'rm -f "$tmp"' EXIT
```

- Register cleanup with trap early.

Publish pattern

```
cmd > "$tmp" && mv "$tmp" final.txt
```

9 Section 9

Batching and File Streams

9.1 xargs builds commands from stdin

- xargs reads items and appends them as command arguments.

Echo items

```
printf 'a\nb\n' | xargs echo
```

- It is useful when a command does not read stdin itself.

Remove listed files

```
printf 'old.tmp\n' | xargs rm
```

9.2 xargs -n controls batch size

- -n limits how many input items go into each command.

One item each

```
printf 'a\nb\n' | xargs -n 1 echo
```

- Small batches are easier to observe and debug.

Two items each

```
printf 'a\nb\nc\n' | xargs -n 2 echo
```

9.3 Use find -print0 with xargs -0

- Whitespace and newlines can appear in filenames.

Safe remove

```
find . -name '*.tmp' -print0 | xargs -0 rm
```

- -print0 separates names with a null byte.

Safe count

```
find . -type f -print0 | xargs -0 wc -l
```

9.4 xargs -I places items inside commands

- -I defines a placeholder token.

Copy by placeholder

```
printf 'a.txt\n' | xargs -I{} cp {} backup/
```

- It runs one command per input item by default.

Echo labels

```
printf 'api\nweb\n' | xargs -I{} echo service={}
```

9.5 Parallel execution needs caution

- Parallel jobs can speed up independent work.

xargs parallelism

```
printf 'a\nb\n' | xargs -n1 -P2 echo
```

- They can also overload disks, APIs, or logs.

GNU parallel style

```
printf 'a\nb\n' | parallel echo {}
```

9.6 cat concatenates files

- cat is ideal for joining files in order.

Show files

```
cat part1.txt part2.txt
```

- It writes file contents to stdout.

Combine files

```
cat part*.txt > combined.txt
```

9.7 split breaks files into pieces

- split writes chunks with generated suffixes.

Split by lines

```
split -l 1000 big.log chunk_
```

- -l splits by line count.

Split by bytes

```
split -b 10M archive.bin part_
```

9.8 dd copies bytes carefully

- dd copies from if= to of= using block sizes.

Copy first block

```
dd if=input.bin of=first.bin bs=1M count=1
```

- It is common for device images and byte-limited copies.

Show progress

```
dd if=image.iso of=/dev/sdX bs=4M status=progress
```

10

Section 10

Filesystem Helpers

10.1 install -m sets mode while copying

- install copies files like cp with extra controls.

Install executable

```
install -m 755 script.sh /usr/local/bin/script
```

- -m sets the destination permissions.

Install config

```
install -m 644 app.conf /etc/app.conf
```

10.2 install -D creates parent directories

- -D creates missing destination directories.

Create parents

```
install -D -m 644 app.conf build/etc/app.conf
```

- It copies one file into the final path.

Install binary path

```
install -D -m 755 tool build/bin/tool
```

10.3 ln -s creates symbolic links

- A symbolic link points to another path by name.

Create link

```
ln -s releases/v1 current
```

- Links are useful for stable names and version switches.

Link a command

```
ln -s /opt/tool/bin/tool ~/bin/tool
```

10.4 readlink -f resolves a path

- readlink -f follows symlinks to a canonical path.

Resolve link

```
readlink -f current
```

- It also resolves . and .. components.

Resolve script path

```
readlink -f "$0"
```

10.5 realpath prints canonical paths

- realpath resolves a path to an absolute form.

Canonical path

```
realpath ../data/../../data/file.txt
```

- It is often clearer than manual cd and pwd tricks.

Store path

```
root=$(realpath .)
echo "$root"
```

10.6 basename extracts the final path component

- basename removes leading directory parts.

Get filename

```
basename /var/log/app.log
```

- It can also strip a suffix.

Strip suffix

```
basename report.txt .txt
```

10.7 dirname extracts the parent path

- `dirname` removes the final path component.

Get parent

```
dirname /var/log/app.log
```

- It returns the directory portion of a path.

Create parent

```
out=build/logs/app.log  
mkdir -p "$(dirname "$out")"
```

10.8 pushd and popd manage directory changes

- pushd changes directory and saves the old location.

Enter and return

```
pushd /tmp  
popd
```

- popd returns to the previous saved location.

Work elsewhere

```
pushd src  
make  
popd
```

11

Section 11

Disk Awareness and Checkpoint

11.1 dirs shows the directory stack

- dirs prints locations saved by pushd.

[Show stack](#)

```
dirs
```

- The leftmost entry is the current directory.

[Clear stack](#)

```
dirs -c
```

11.2 du -sh summarizes path size

- du reports disk usage for files and directories.

Directory size

```
du -sh logs/
```

- -s summarizes each argument.

Several paths

```
du -sh build dist cache
```

11.3 df -h shows filesystem free space

- df reports space at the filesystem level.

All filesystems

```
df -h
```

- -h prints sizes in human-friendly units.

One path

```
df -h .
```

11.4 pv previews pipeline progress

- pv can show throughput and progress for stream data.

Show copy progress

```
pv image.iso > /tmp/image.iso
```

- It is optional and may not be installed everywhere.

Progress in pipe

```
tar cf - dir | pv | gzip > dir.tar.gz
```

11.5 Compare common redirect mistakes

- > replaces while >> appends.

Append instead of replace

```
echo next >> app.log
```

- 2>&1 depends on the stdout target at that point.

Merge before pipe

```
cmd 2>&1 | tee all.log
```

11.6 Choose log locations deliberately

- Logs should go somewhere predictable.

Create log dir

```
mkdir -p logs  
./job.sh >> logs/job.log 2>&1
```

- Create log directories before redirecting into them.

Absolute log path

```
log="$HOME/logs/job.log"  
./job.sh >> "$log" 2>&1
```

11.7 Review files and pipes

- Use streams to connect commands cleanly.

Clean pipeline

```
grep ERROR app.log | sort | uniq -c
```

- Choose redirects based on which descriptor should move.

Safe capture

```
./job.sh > job.out 2> job.err
```

11.8 Module 05 checkpoint

- Explain where stdout, stdin, and stderr are going.

Checkpoint pipeline

```
find . -type f -print0 | xargs -0 wc -l
```

- Build a pipeline that saves and displays output.

Checkpoint cleanup

```
tmp=$(mktemp)  
trap 'rm -f "$tmp"' EXIT
```

NEXT STEP

Practice every example in your terminal

Then open the next module deck

Merit Advisory
05 · Files & Pipes

